



NMRSpinDynamics

spin-1/2 simulations in inhomogeneous fields

MATLAB reference and Python port

PythonSpinDynamics

User Manual

Python port of MATLABSpinDynamics

Simulation workflows for spin-1/2 nuclear magnetization in possibly non-uniform and time-varying magnetic fields, with validated MATLAB-compatible Bloch kernels, Python-native analysis tools, and scoped scalar-coupled spin-1/2 extensions.

June 2026

Contents

1	Physical Scope and Assumptions	1
1.1	Spin Bath Cartoon	1
1.2	Consequences for Users	1
2	Overview	3
2.1	Package Layout	3
3	Installation and Test Runs	4
3.1	Editable Install	4
3.2	Tests	4
4	Model Conventions	5
4.1	Coherence Ordering	5
4.2	Normalized Time	5
4.3	Pulse Timing Cartoon	5
4.4	Effective Rotations	5
4.5	Rotation Cartoon	6
4.6	Relaxation	6
4.7	Echo Conversion	6
4.8	Radiation Damping	6
5	Scalar J-Coupling Models	8
5.1	J-Coupled Spin Cartoon	8
5.2	Dense Spin-1/2 Hamiltonians	8
5.3	Inhomogeneous B0/B1 Isochromats	9
5.4	Low-Field J-Editing	10
5.5	TANGO-B Filtering	11
5.6	SLIC Response	11
6	First Workflows	13
6.1	CPMG	13
6.2	Finite Echo Trains	13
6.3	FID	13

6.4	Imaging	13
6.5	Diffusion	14
6.6	Time-Varying Fields	14
6.7	WURST	15
6.8	Radiation Damping	15
6.9	Inverse Laplace Analysis	15
7	Examples	17
8	Validation	18
9	API Reference	19
9.1	Workflow Results	19
9.2	High-Level Workflows	19
9.3	Imaging API	20
9.4	Parameter Constructors	20
9.5	Analysis API	21
9.6	Core Numerical API	21
9.7	Probe and Pulse API	22
9.8	Noise, Motion, and Radiation Damping	23
9.9	Scalar Coupling API	23
9.10	Optimization API	25
10	Known Gaps	26

1 Physical Scope and Assumptions

The MATLAB-compatible workflow layer in PythonSpinDynamics models a bath of uncoupled spin-1/2 nuclei evolving in a static, spatially non-uniform, or time-varying main field $B_0(\mathbf{r}, t)$ and applied RF fields. The elementary object in that layer is an isochromat: a subensemble of spin-1/2 nuclei that shares an offset, RF sensitivity, relaxation constants, and probe response.

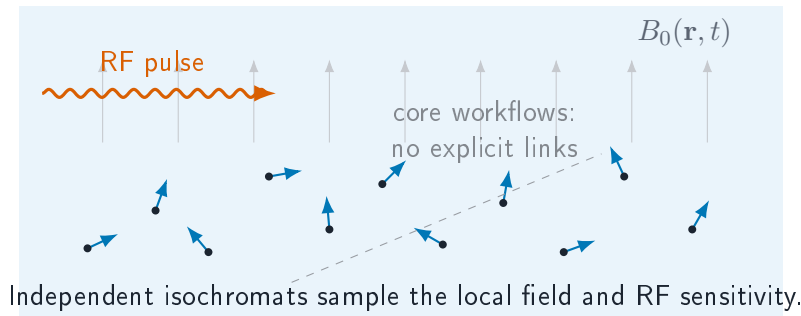
Modeled directly in the core Bloch workflows. Independent spin-1/2 magnetization vectors; offset distributions from $B_0(\mathbf{r}, t)$; RF rotations; free precession; phenomenological T_1 and T_2 relaxation; probe transmit and receive filtering; optional deterministic radiation damping; optional motion through field maps.

Scalar-coupling extension. The `spin_dynamics.coupling` namespace adds small dense spin-1/2 systems with scalar J couplings, analytic low-field J-editing models, ideal TANGO-B filter curves, and initial SLIC simulations. These models are intentionally scoped and are documented in Chapter 5.

Still outside the package scope. Dipolar coupling networks, chemical exchange, quadrupolar spins with $I > 1/2$, arbitrary multi-quantum pulse sequences, shaped sample microstructure beyond the supplied field and density maps, microscopic spin noise, and large sparse coupled-spin simulations. Spin-spin effects enter the Bloch workflows only indirectly when represented by effective relaxation times such as T_2 or T_2^* .

This scope is deliberate. The main package follows MATLAB's classical Bloch/isochromat viewpoint, which is appropriate for many pulse, probe, imaging, and relaxation studies where the spins can be treated as independent subensembles. The coupled-spin namespace adds targeted quantum spin-1/2 tools for scalar J-coupling phenomena without turning the full package into a general quantum spin Hamiltonian framework.

1.1 Spin Bath Cartoon



1.2 Consequences for Users

- Use offset grids, field maps, or moving particles to represent spatially varying B_0 , transmit B_1 , and receive B_1 effects.

- Use relaxation constants to represent ensemble dephasing and energy exchange that are outside the explicit isochromat model.
- Use `spin_dynamics.coupling` when a small spin-1/2 scalar J-coupled model is the phenomenon of interest.
- Do not use the package for quadrupolar transitions, chemical exchange, large coupled-spin networks, or arbitrary multi-quantum coherence pathways.

2 Overview

PythonSpinDynamics is a Python port of the MATLAB spin-dynamics simulation package. The MATLAB tree under `../MATLABSpinDynamics/SpinDynamicsUpdated/Version_2/code` remains the reference for numerical behavior, while this workspace provides a typed, testable Python API for simulations, validation fixtures, examples, and new analysis workflows.

The supported surface currently includes ideal, tuned, untuned, and matched probe CPMG workflows; finite echo trains; FID; phase-encoded imaging; diffusion CPMG; time-varying-field CPMG; WURST inversion and matched WURST-CPMG; radiation damping; moving-isochromat motion helpers; received-signal noise; inverse-Laplace analysis; compact pulse-optimization scaffolds; and small spin-1/2 scalar-coupling tools for low-field J-editing, TANGO-B filtering, and SLIC-style spin-lock response.

2.1 Package Layout

Path	Purpose
<code>src/spin_dynamics/</code>	Installable Python package.
<code>examples/</code>	CLI examples and plotting scripts.
<code>tests/</code>	Smoke tests, validation tests, and unit tests.
<code>validation/fixtures/</code>	MATLAB/Octave fixture arrays.
<code>validation/octave/</code>	Fixture generation scripts.
<code>docs/python_api/</code>	Lightweight Markdown API notes.
<code>docs/validation_results.md</code>	Current validation status and tolerance notes.
<code>docs/radiation_damping.md</code>	Focused radiation-damping model notes.

3 Installation and Test Runs

3.1 Editable Install

PythonSpinDynamics requires Python 3.10 or newer. The core install depends on NumPy 1.24 or newer and does not require MATLAB at runtime. MATLAB is only needed to regenerate the complete MATLAB reference fixture set; Octave can regenerate the smaller dependency-light fixtures.

From PythonSpinDynamics, install the package into an active environment:

```
python -m pip install -e .
```

Optional extras are provided for development, plotting, benchmarking, and SciPy-backed optimization:

```
python -m pip install -e ".[dev,opt,plot,bench]"
```

The core package depends only on NumPy. The `opt` extra enables SciPy-backed continuous optimizers, non-negative inverse-Laplace solves, and MATLAB `.mat` result import/export helpers. The `plot` extra enables the Matplotlib examples.

3.2 Tests

Use the smoke tier during normal editing:

```
python -m unittest tests.smoke_tests
```

Run the full validation suite before committing numerical or workflow changes:

```
python -m unittest discover -s tests
```

4 Model Conventions

4.1 Coherence Ordering

Low-level kernels preserve MATLAB's coherence ordering:

$$\mathbf{M} = [M_0 \quad M_- \quad M_+]^T.$$

This ordering differs from many Bloch-vector presentations, so new kernels and validation tests should state the expected ordering explicitly.

4.2 Normalized Time

The core MATLAB routines usually normalize time to the nominal RF amplitude $\omega_1 = 1$. In these units, a 90-degree hard pulse has duration

$$t_{90} = \frac{\pi}{2},$$

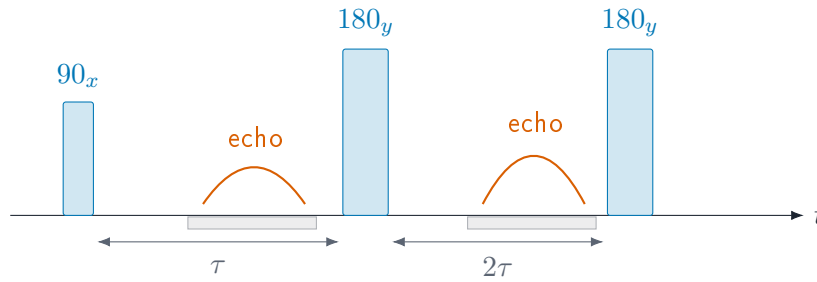
and a 180-degree hard pulse has duration

$$t_{180} = \pi.$$

Workflow helpers that expose physical seconds convert into these normalized units before calling the lower-level kernels.

4.3 Pulse Timing Cartoon

A minimal spin-echo or CPMG building block is a sequence of RF pulses, free precession windows, and acquisition windows. The package represents these as ordered intervals with pulse matrices and free-precession delays.



4.4 Effective Rotations

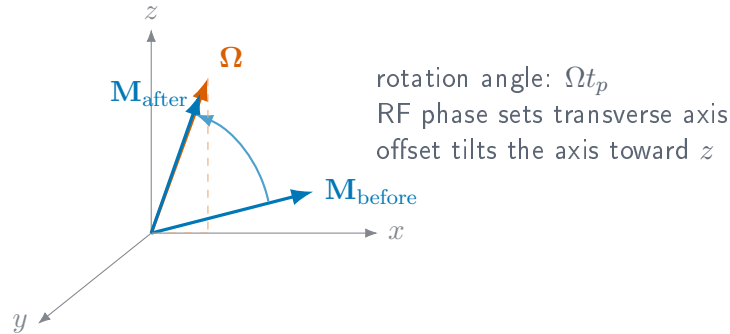
For an isochromat with normalized offset $\Delta\omega$, RF amplitude ω_1 , phase ϕ , and pulse duration t_p , the rotating-frame effective field is

$$\mathbf{\Omega} = \begin{bmatrix} \omega_1 \cos \phi \\ \omega_1 \sin \phi \\ \Delta\omega \end{bmatrix}, \quad \Omega = \sqrt{\omega_1^2 + \Delta\omega^2}.$$

The magnetization update is a rotation by angle Ωt_p about Ω/Ω . The Python rotation helpers expose this calculation through `spin_dynamics.core.rotations` and return array shapes chosen to match the MATLAB reference fixtures.

4.5 Rotation Cartoon

In the rotating frame, a rectangular RF segment rotates the magnetization about an effective axis set by RF phase, RF amplitude, and off-resonance offset.



4.6 Relaxation

Free-precession intervals with relaxation use the usual exponential factors:

$$M_{xy}(t + \Delta t) = M_{xy}(t) \exp\left(-\frac{\Delta t}{T_2}\right) \exp(-i\Delta\omega \Delta t),$$

$$M_z(t + \Delta t) = M_{th} + (M_z(t) - M_{th}) \exp\left(-\frac{\Delta t}{T_1}\right).$$

Finite acquisition workflows assemble these intervals from pulse-sequence parameters and then call the `arb10`-style kernels.

4.7 Echo Conversion

The echo utilities direct-sum a received spectrum $S(\Delta\omega)$ over the offset grid:

$$e(t) = \sum_k S(\Delta\omega_k) \exp(i\Delta\omega_k t) \Delta\omega.$$

The implementation uses the same direct-sum convention as the MATLAB `calc_time_domain_echo` reference path. FID traces use the corresponding FID conversion helper.

4.8 Radiation Damping

The deterministic radiation-damping back-action model follows the local Measurements text convention:

$$T_{rd} = \frac{2}{\gamma\mu_0\eta M_0 Q},$$

where γ is the gyromagnetic ratio, η is the magnetic-energy fill factor, M_0 is the equilibrium magnetization density, and Q is the loaded probe quality factor.

The analytic on-resonance hard-pulse FID envelope used by the test suite is

$$|M_{xy}(t)| = \frac{M_0}{\cosh(t/T_{\text{rd}} - \log \tan(\theta/2))}.$$

The finite-sequence implementation can add feedback during free-precession and acquisition windows, and can optionally apply an operator-split approximation during RF pulse matrices.

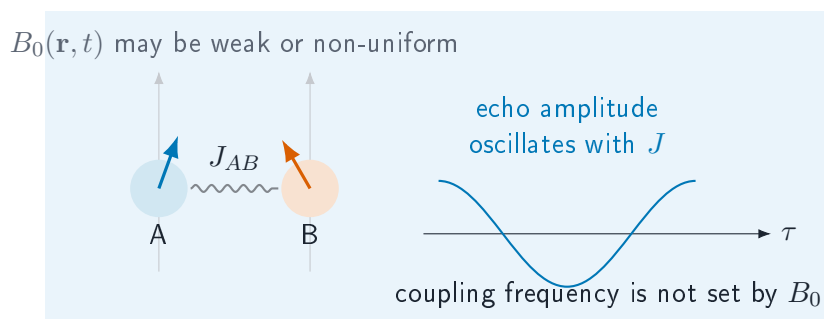
5 Scalar J-Coupling Models

Scalar J coupling is field independent, so it can remain visible in weak or grossly inhomogeneous magnetic fields where chemical-shift resolution is compressed. The `spin_dynamics.coupling` namespace adds a scoped set of spin-1/2 tools for these cases. It is separate from the MATLAB-compatible Bloch workflow layer: Bloch workflows propagate independent isochromats, while the coupling layer propagates small dense Hilbert-space systems or evaluates closed-form J-editing response curves.

Use this layer for: heteronuclear low-field J-editing curves, known-J spectrum fitting, ideal TANGO-B filter selectivity, and exploratory two-spin or few-spin scalar-coupled Hamiltonian simulations.

Current limits: spin-1/2 only; dense matrices only; no relaxation superoperators; no chemical exchange; no quadrupolar nuclei; no large sparse coupled-spin networks; no full pulse-sequence compiler for arbitrary multi-quantum pathways.

5.1 J-Coupled Spin Cartoon



5.2 Dense Spin-1/2 Hamiltonians

A coupled system is built from offsets ν_i in hertz and symmetric scalar couplings J_{ij} in hertz:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    isotropic_j_hamiltonian,
    zeeman_hamiltonian,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
    labels=["A", "B"],
)

hamiltonian = zeeman_hamiltonian(system) + isotropic_j_hamiltonian(system)
```

Hamiltonian builders return angular-frequency matrices in radians per second and use spin-1/2 operators

with eigenvalues $\pm 1/2$. The rotating-frame offset Hamiltonian is

$$H_Z = 2\pi \sum_i \nu_i I_{iz}.$$

For weakly coupled systems, the secular scalar Hamiltonian is

$$H_J^{\text{sec}} = 2\pi \sum_{i < j} J_{ij} I_{iz} I_{jz},$$

while the isotropic scalar Hamiltonian is

$$H_J^{\text{iso}} = 2\pi \sum_{i < j} J_{ij} (I_{ix} I_{jx} + I_{iy} I_{jy} + I_{iz} I_{jz}).$$

The dense propagator evaluates

$$\rho(t) = U(t)\rho(0)U^\dagger(t), \quad U(t) = \exp(-iHt),$$

and is intended for small systems where dense matrices are acceptable.

5.3 Inhomogeneous B0/B1 Isochromats

The bridge between the coupled-spin layer and the Bloch workflow layer is a coupled isochromat ensemble. Each isochromat contains the same scalar-coupled spin network, but samples a different local field and receive weight. For isochromat k , the local spin offsets are

$$\nu_{i,k} = \nu_i^{(0)} + s_i \Delta\nu_k,$$

where $\nu_i^{(0)}$ is the base spin offset, $\Delta\nu_k$ is the local B0 offset in hertz, and s_i is a per-spin offset scale. The default $s_i = 1$ adds the same field offset to every spin. Heteronuclear or carrier-specific models can supply different scales.

During an RF or spin-lock interval, the nominal nutation frequency is scaled by the local transmit field:

$$\omega_{1,k} = b_{1,k}^{\text{tx}} \omega_1.$$

The ensemble signal is accumulated as

$$S = \sum_k w_k b_{1,k}^{\text{rx}} \text{Tr}(\rho_k D),$$

where w_k is the isochromat weight, $b_{1,k}^{\text{rx}}$ is the local receive scale, and D is the detection operator.

```
from spin_dynamics.coupling import (
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
)

ensemble = coupled_isochromat_ensemble(
    system,
    b0_offsets_hz=[-2.0, 0.0, 2.0],
```

```

    weights=[0.25, 0.5, 0.25],
    b1_tx_scale=[0.9, 1.0, 1.1],
    b1_rx_scale=[1.0, 1.0, 0.8],
)

result = simulate_coupled_isochromat_sequence(
    ensemble,
    [
        rf_step(duration=0.25, nutation_hz=1.0, phase=1.5708),
        free_precession_step(duration=0.01),
    ],
    initial_axis="x",
    detect_axis="x",
)

```

For time-varying fields, each sequence step may override the ensemble B0 offsets or B1 transmit scale. This keeps the first implementation close to the existing isochromat approach: the coupled-spin Hilbert space is dense and small, while field inhomogeneity is represented by an outer ensemble sum.

5.4 Low-Field J-Editing

Analytic J-editing helpers model response curves as sums of field-independent cosine modulations:

$$s(\tau) = b + \sum_m a_m \cos^{p_m}(2\pi N J_m \tau),$$

where τ is the encoding time, N is the number of encoding cycles, J_m is a known or hypothesized coupling in hertz, a_m is its amplitude, and p_m is an optional multiplicity exponent. Proton-detected models use $p_m = 1$. Carbon-detected CH_n groups use $p_m = n$.

```

from spin_dynamics.coupling import (
    carbon_detected_j_modulation,
    fit_known_j_spectrum,
    j_modulation_curve,
)

signal = j_modulation_curve(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    amplitudes=[0.85, 0.15],
)

carbon_signal = carbon_detected_j_modulation(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    abundances=[0.85, 0.15],
    proton_counts=[2, 1],
)

fit = fit_known_j_spectrum(
    encoding_times,
    signal,
    couplings_hz=[125.0, 160.0],
    include_background=False,
)

```

```
)
```

`fit_known_j_spectrum` solves a linear least-squares problem once the candidate J_m values are supplied. It returns amplitudes, optional background, fitted signal, residual, rank, and residual norm.

5.5 TANGO-B Filtering

The ideal TANGO-B helper evaluates the coupled-spin transverse filter envelope

$$A(J; t_d) = \sin^2(\pi J t_d).$$

When a target coupling J_* and positive odd order n are supplied, the delay is selected as

$$t_d = \frac{n}{2J_*}.$$

```
from spin_dynamics.coupling import tango_b_filter

response = tango_b_filter(
    couplings_hz,
    target_coupling_hz=160.0,
    order=3,
)
```

This model is idealized: it reports the scalar-coupling selectivity envelope, not RF imperfections, relaxation, diffusion, or probe filtering.

5.6 SLIC Response

Spin-lock induced crossing (SLIC) is modeled by adding an RF spin-lock Hamiltonian to the static coupled-spin Hamiltonian:

$$H_{\text{SLIC}} = H_Z + H_J^{\text{iso}} + 2\pi\omega_1 \sum_i I_{ix}.$$

The helper scans spin-lock nutation frequencies ω_1 in hertz and returns the remaining transverse magnetization and its corresponding dip:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    simulate_slic_spectrum,
    two_spin_slic_transfer_time,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
)
duration = two_spin_slic_transfer_time(offset_difference_hz=0.7)
result = simulate_slic_spectrum(
    system,
    nutation_frequencies_hz,
    spin_lock_time=duration,
)
```

For a nearly equivalent two-spin system, the SLIC dip is expected near the coupling frequency. The helper is exploratory and dense; broad homonuclear networks will need sparse or specialized methods.

6 First Workflows

6.1 CPMG

Application code should normally start with the public workflow runners:

```
from spin_dynamics.workflows import run_tuned_cpmg

result = run_tuned_cpmg(numpts=101, maxoffs=10)
print(result.del_w.shape)
print(result.echo.shape)
print(result.snr)
```

The asymptotic CPMG runners return CPMGResult objects with the offset grid, asymptotic magnetization, received spectrum, time-domain echo, acquisition time vector, optional SNR, and probe label.

6.2 Finite Echo Trains

```
from spin_dynamics.workflows import run_matched_cpmg_train

train = run_matched_cpmg_train(
    numpts=51,
    num_echoes=8,
    auto_refine_grid=True,
    num_workers=None,
)
```

Finite train results contain one acquired spectrum and one echo per echo index. They also include trapezoidal echo integrals and physical echo-center times. For long trains, use the rephasing checks or enable `auto_refine_grid=True` so the offset grid is refined before RF matrices are built.

6.3 FID

```
from spin_dynamics.parameters import set_params_ideal_fid
from spin_dynamics.workflows.fid import sim_fid_ideal

sp, pp = set_params_ideal_fid(numpts=101)
macq, fid, tvect = sim_fid_ideal(sp, pp)
```

6.4 Imaging

```
import numpy as np
from spin_dynamics.workflows import (
    make_imaging_field_maps,
    run_tuned_phase_encoded_cpmg_imaging,
```



```

        form_imaging_image,
    )

    rho = np.eye(8)
    fields = make_imaging_field_maps(
        rho,
        b0_map=np.zeros_like(rho),
        b1_tx_map=np.ones_like(rho),
        b1_rx_map=np.ones_like(rho),
    )
    result = run_tuned_phase_encoded_cpmg_imaging(
        fields,
        num_echoes=4,
        ny=9,
        receive_mode="weighted",
    )
    image = form_imaging_image(result, mode="echo_sum")

```

Imaging workflows return raw k-space, reconstructed per-echo images, field maps, relaxation maps, gradient metadata, and sequence timing. Image formation is intentionally separated from acquisition so selected-echo, echo-summed, fitted- ρ , and fitted- T_2 displays can be compared.

6.5 Diffusion

```

from spin_dynamics.workflows import run_matched_diffusion_q_sweep

sweep = run_matched_diffusion_q_sweep(
    q_values=[20, 50],
    num_echoes=3,
    numpts=21,
    auto_refine_grid=True,
)

```

The compact matched-probe diffusion workflow is solver-validated through $Q = 2000$. Higher- Q cases remain a stiffness target for the current NumPy matched-probe transient solver.

6.6 Time-Varying Fields

```

from spin_dynamics.workflows import (
    sinusoidal_field_waveform,
    run_ideal_time_varying_amplitude_sweep,
)

waveform = sinusoidal_field_waveform(num_echoes=16)
sweep = run_ideal_time_varying_amplitude_sweep(
    amplitudes=[0, 0.5, 1.0, 2.0],
    waveform=waveform,
    numpts=101,
    auto_refine_grid=True,
)

```

6.7 WURST

```
from spin_dynamics.workflows import (
    run_ideal_wurst_inversion,
    run_matched_wurst_cpmg,
)

ideal = run_ideal_wurst_inversion(numpts=101)
matched = run_matched_wurst_cpmg(num_echoes=2, numpts=51, num_steps=64)
```

6.8 Radiation Damping

```
from spin_dynamics.workflows import run_radiation_damping_fid

fid = run_radiation_damping_fid(
    probe="matched",
    fill_factor=0.7,
    equilibrium_magnetization=0.8,
    flip_angle=1.0,
    model="circuit",
    detuning=2.0e4,
)
```

Finite tuned and matched CPMG trains accept an opt-in radiation_damping mapping:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=51,
    num_echoes=8,
    radiation_damping={
        "fill_factor": 0.7,
        "equilibrium_magnetization": 0.8,
        "model": "circuit",
        "detuning": 2.0e4,
        "apply_during_pulses": True,
    },
)
```

6.9 Inverse Laplace Analysis

```
import numpy as np
from spin_dynamics.analysis import invert_t2, t2_kernel

echo_times = np.linspace(0.5e-3, 90e-3, 40)
t2_axis = np.logspace(-4, -1, 60)
signal = t2_kernel(echo_times, t2_axis) @ np.exp(-((np.log(t2_axis) + 4.5) ** 2))

result = invert_t2(
    signal,
```

```
    echo_times,  
    t2_axis,  
    regularization=5e-4,  
    regularization_order=2,  
)
```

For two-dimensional distributions, the separable forward model is

$$D = K_1 F K_2^T,$$

where D is the measured data, F is the unknown distribution, and K_1 , K_2 are relaxation or diffusion kernels. Use `invert_t1_t2` or `invert_d_t2` for common NMR cases.

7 Examples

The example scripts can be run from PythonSpinDynamics; each script also works from inside examples/ by adding the local source tree to `sys.path` when needed.

```
python examples\ideal_cpmg.py --numpts 101
python examples\ideal_fid.py --numpts 101
python examples\ideal_cpmg_train.py --numpts 101 --num-echoes 8
python examples\probe_cpmg_compare.py --numpts 101
python examples\finite_probe_train_sweeps.py --numpts 21 --num-echoes 3
python examples\matched_diffusion_cpmg.py --numpts 21 --num-echoes 3
python examples\radiation_damping_fid.py --probe matched --points 401
python examples\nmr_maser.py
python examples\heteronuclear_j_editing.py --points 33
python examples\coupled_isochromat_fields.py --points 21
```

Plotting examples require Matplotlib:

```
python examples\plot_ideal_workflows.py --numpts 201 --output results\ideal_workflows.png
python examples\plot_ideal_imaging.py --pixels 6 --ny 7 --output results\ideal_imaging.png
python examples\plot_inverse_laplace.py --output results\inverse_laplace.png
python examples\plot_radiation_damping.py --output results\radiation_damping.png
python examples\plot_wurst_flow.py --numpts 41 --num-steps 64 --output results\wurst_flow.
    png
python examples\plot_j_editing_spectrum.py --output results\j_editing_spectrum.png
python examples\plot_j_editing_field_spread.py --output results\j_editing_field_spread.png
python examples\plot_tango_filter.py --target 160 --output results\tango_filter.png
python examples\plot_slic_two_spin.py --j-hz 7 --delta-hz 0.7 --output results\
    slic_two_spin.png
```

8 Validation

Validation fixtures live in `validation/fixtures`. The tests compare the Python implementation against compact MATLAB or Octave arrays for core rotations, echo conversion, FID, the `arb10` kernel, finite acquisition, probe response, imaging, pulse-shape helpers, optimization result handling, and public workflow result shapes.

Regenerate dependency-light fixtures with Octave or MATLAB from the `PythonSpinDynamics` directory:

```
matlab -batch "run('validation/octave/generate_basic_fixtures.m')"  
matlab -batch "run('validation/octave/generate_imaging_fixtures.m')"  
matlab -batch "run('validation/octave/generate_optimization_fixtures.m')"  
matlab -batch "run('validation/octave/generate_pulse_fixtures.m')"
```

Matched-probe fixtures require MATLAB toolboxes used by the reference implementation. Octave fixture generation skips matched-probe files when `fmincon` is unavailable.

9 API Reference

This chapter documents the intended public surface. Lower-level module imports remain available for validation and porting. For application code, start with:

```
spin_dynamics.workflows
spin_dynamics.parameters
spin_dynamics.analysis
spin_dynamics.coupling
```

Drop into lower-level modules only when a building block is needed directly. Use `spin_dynamics.coupling` for the scoped dense scalar-coupled spin-1/2 tools described in Chapter 5.

9.1 Workflow Results

Class	Purpose
CPMGResult	Asymptotic CPMG spectrum, echo, time vector, SNR, and probe label.
CPMGTrainResult	Finite echo-train spectra, echoes, echo integrals, and sequence timing.
CPMGIRTrainResult	Inversion-recovery finite trains over a tauvect.
CPMGParameterSweepResult	Asymptotic Q or mistuning sweeps.
CPMGFiniteParameterSweepResult	Finite-train Q or mistuning sweeps.
ZMagnetizationSweepResult	Matched-probe z-magnetization Q sweeps.
IdealTimeVaryingCPMGResult	Final echo for ideal time-varying-field CPMG.
ProbeTimeVaryingCPMGResult	Probe-aware time-varying-field CPMG result.
IdealCPMGImagingResult	Ideal phase-encoded CPMG imaging result.
ProbeCPMGImagingResult	Tuned or matched phase-encoded CPMG imaging result.
MatchedDiffusionCPMGResult	Matched diffusion-aware CPMG train.
MatchedDiffusionQSweepResult	Matched diffusion Q sweep.
WURSTInversionResult	WURST inversion profile and pulse metadata.
MatchedWURSTCPMGResult	Matched WURST-CPMG echoes and spectra.
RadiationDampingFIDResult	Radiation-damping FID simulation and analytic comparison fields.

9.2 High-Level Workflows

```
from spin_dynamics.workflows import (
    run_ideal_cpmg, run_tuned_cpmg, run_untuned_cpmg, run_matched_cpmg,
    run_ideal_cpmg_train, run_tuned_cpmg_train,
    run_untuned_cpmg_train, run_matched_cpmg_train,
    run_ideal_cpmg_ir_train, run_tuned_cpmg_ir_train,
    run_untuned_cpmg_ir_train, run_matched_cpmg_ir_train,
    run_tuned_q_sweep, run_matched_q_sweep,
```

```

run_tuned_mistuning_sweep, run_matched_mistuning_sweep,
run_tuned_finite_q_sweep, run_untuned_finite_q_sweep,
run_matched_finite_q_sweep,
run_tuned_finite_mistuning_sweep,
run_untuned_finite_mistuning_sweep,
run_matched_finite_mistuning_sweep,
run_matched_z_magnetization_q_sweep,
run_matched_diffusion_cpmg, run_matched_diffusion_q_sweep,
run_ideal_time_varying_cpmg_final,
run_tuned_time_varying_cpmg_final,
run_untuned_time_varying_cpmg_final,
run_matched_time_varying_cpmg_final,
run_ideal_time_varying_amplitude_sweep,
run_tuned_time_varying_amplitude_sweep,
run_untuned_time_varying_amplitude_sweep,
run_matched_time_varying_amplitude_sweep,
sinusoidal_field_waveform,
run_ideal_wurst_inversion,
run_matched_wurst_inversion,
run_matched_wurst_cpmg,
run_radiation_damping_fid,
)

```

The main CPMG runners share `numpts` and `maxoffs`. Finite train runners add `num_echoes`, relaxation constants, rephasing controls, optional `auto_refine_grid`, and optional worker controls. Probe-aware train runners also accept `q_value` and `mistuning_offset` where the underlying probe model supports them.

9.3 Imaging API

```

from spin_dynamics.workflows import (
    ImagingFieldMaps,
    make_imaging_field_maps,
    load_imaging_field_maps_npz,
    run_ideal_phase_encoded_cpmg_imaging,
    run_tuned_phase_encoded_cpmg_imaging,
    run_matched_phase_encoded_cpmg_imaging,
    run_t1_encoded_phase_encoded_cpmg_imaging,
    reconstruct_image_from_kspace,
    form_imaging_image,
    fit_imaging_echo_decay,
    summarize_imaging_noise_trials,
)

```

Prefer the `phase_encoded` names for new code. The shorter compatibility aliases remain available:

```

run_ideal_cpmg_imaging
run_tuned_cpmg_imaging
run_matched_cpmg_imaging

```

9.4 Parameter Constructors

```

from spin_dynamics.parameters import (
    set_params_ideal,
    set_params_ideal_fid,
    set_params_tuned_orig,
    set_params_untuned_orig,
    set_params_matched_orig,
    set_params_tuned_spa,
    set_params_untuned_spa,
    set_params_matched_spa,
    set_params_tuned_jmr,
    set_params_untuned_jmr,
    set_params_matched_jmr,
)

```

These constructors return dataclass instances mirroring MATLAB `sp`, `pp`, and `params` structures. They accept optional `numpts` arguments for lightweight tests and examples while preserving MATLAB defaults when omitted.

9.5 Analysis API

```

from spin_dynamics.analysis import (
    t2_kernel, t1_kernel, diffusion_kernel, laplace_kernel,
    invert_laplace_1d, invert_laplace_2d,
    invert_t2, invert_t1, invert_t1_t2, invert_d_t2,
    default_regularization_strengths,
    estimate_noise_rms_from_snr,
    expected_residual_norm_from_snr,
    select_regularization_1d,
    select_regularization_2d,
)

```

For one-dimensional relaxation distributions, use:

```

invert_t2
invert_t1

```

For separable two-dimensional maps, use:

```

invert_t1_t2
invert_d_t2

```

The `select_regularization_*` helpers choose Tikhonov strengths from an RMS SNR estimate.

9.6 Core Numerical API

```

from spin_dynamics.core.echo import (
    calc_time_domain_echo,
    calc_time_domain_echo_arb,
    calc_fid_time_domain,
)
from spin_dynamics.core.rotations import (
    calc_rotation_matrix,
)

```



```

    calc_rot_axis_arba3,
    calc_rot_axis_arba4,
    calc_v0crit,
    sim_spin_dynamics_asymp_mag3,
    sim_spin_dynamics_exc,
)
from spin_dynamics.core.kernels import (
    sim_spin_dynamics_arb7,
    sim_spin_dynamics_arb10,
    sim_spin_dynamics_arb10_chunked,
    sim_spin_dynamics_arb10_diffusion,
    sim_spin_dynamics_arb10_radiation_damping,
)
from spin_dynamics.core.isochromats import (
    analyze_rephasing,
    check_rephasing,
    estimate_rephase_time,
    recommended_numpts_for_rephasing,
)

```

Core functions are intended for validation, debugging, and continued porting. They are closer to MATLAB's array contracts than the workflow layer.

9.7 Probe and Pulse API

```

from spin_dynamics.probes.tuned import (
    tuned_probe_lp,
    tuned_probe_rx,
    tuned_probe_rx_tf,
    calc_rot_axis_tuned_probe_lp_orig2,
    calc_masy_tuned_probe_lp_orig,
)
from spin_dynamics.probes.untuned import (
    untuned_probe_lp,
    untuned_probe_rx,
    untuned_probe_rx_tf,
    calc_rot_axis_untuned_probe_lp,
    calc_masy_untuned_probe_lp,
)
from spin_dynamics.probes.matched import (
    matching_network_design2,
    find_coil_current,
    find_coil_current_wurst,
    matched_probe_rx,
    calc_rot_axis_matched_probe,
    calc_masy_matched_probe_orig,
    calc_masy_matched_nut,
)
from spin_dynamics.pulses import (
    quantize_phase,
    create_wurst_pulse,
    adjust_untuned_segment_lengths,
    tuned_rectangular_pulse_response,
)

```

```
untuned_rectangular_pulse_response,  
matched_rectangular_pulse_response,  
matched_wurst_pulse_response,  
)
```

9.8 Noise, Motion, and Radiation Damping

```
from spin_dynamics.noise import (  
    NoiseSpec,  
    add_received_noise,  
    estimate_matched_filter_snr,  
    ideal_noise_density,  
    tuned_probe_output_noise_density,  
    untuned_probe_output_noise_density,  
    matched_probe_output_noise_density,  
)  
from spin_dynamics.motion import (  
    make_motion_field_maps_2d,  
    initialize_ensemble_from_density,  
    advect_diffuse_positions,  
    move_ensemble,  
    apply_free_precession,  
    apply_rf_rotation,  
    free_precession_with_motion_step,  
    receive_signal,  
    transverse_b1_magnitude,  
)  
from spin_dynamics.radiation_damping import (  
    radiation_damping_time,  
    proton_thermal_magnetization_density,  
    water_proton_sample,  
    hyperpolarized_proton_sample,  
    normalized_radiation_damping_weights,  
    radiation_damping_probe_from_tuned,  
    radiation_damping_probe_from_matched,  
    initial_state_from_flip_angle,  
    analytic_radiation_damping_envelope,  
    simulate_radiation_damping,  
    simulate_radiation_damping_fid,  
    simulate_nmr_maser,  
)
```

9.9 Scalar Coupling API

```
from spin_dynamics.coupling import (  
    CoupledSpinSystem,  
    CoupledIsochromatEnsemble,  
    CoupledIsochromatStep,  
    CoupledIsochromatSequenceResult,  
    coupled_spin_system,
```

```

coupled_isochromat_ensemble,
free_precession_step,
rf_step,
simulate_coupled_isochromat_sequence,
spin_operator,
total_operator,
product_operator,
zeeman_hamiltonian,
secular_j_hamiltonian,
isotropic_j_hamiltonian,
rf_hamiltonian,
equilibrium_density,
evolve_density,
expectation,
j_modulation_curve,
proton_detected_j_modulation,
carbon_detected_j_modulation,
tango_b_filter,
fit_known_j_spectrum,
JEditingFitResult,
simulate_slic_spectrum,
two_spin_slic_transfer_time,
SLICSpectrumResult,
)

```

Object	Purpose
CoupledSpinSystem	Validated small spin-1/2 system with offsets and scalar couplings in hertz.
CoupledIsochromatEnsemble	Ensemble of local B0/B1 field samples for one coupled-spin system.
free_precession_step, rf_step	Sequence-step builders with optional time-varying B0/B1 overrides.
simulate_coupled_isochromat_sequence	Dense coupled-spin sequence propagation over an isochromat ensemble.
spin_operator, total_operator, product_operator	Dense spin-1/2 operator builders.
zeeman_hamiltonian	Rotating-frame offset Hamiltonian in radians per second.
secular_j_hamiltonian	Weak-coupling $I_z I_z$ scalar Hamiltonian.
isotropic_j_hamiltonian	Isotropic scalar Hamiltonian for strongly coupled spin-1/2 systems.
rf_hamiltonian	RF nutation Hamiltonian for selected spins.
equilibrium_density, evolve_density, expectation	Minimal dense density-matrix utilities.
j_modulation_curve	Sum of cosine J-modulation components.
proton_detected_j_modulation	Proton-detected low-field J-editing response.
carbon_detected_j_modulation	Carbon-detected $\cos^n$ J-editing response for CH_n groups.
tango_b_filter	Ideal TANGO-B scalar-coupling filter envelope.

<code>fit_known_j_spectrum</code>	Least-squares amplitudes for supplied J-coupling positions.
<code>simulate_slic_spectrum</code>	Spin-lock response scan for dense coupled-spin systems.

9.10 Optimization API

```

from spin_dynamics.optimization import (
    spa_pulse_list,
    rectangular_refocusing_lengths,
    evaluate_tuned_refocusing_pulse,
    evaluate_untuned_refocusing_pulse,
    evaluate_matched_refocusing_pulse,
    evaluate_ideal_v0crit_refocusing_pulse,
    evaluate_ideal_v0crit_excited_refocusing_pulse,
    evaluate_ideal_time_varying_refocusing_pulse,
    optimize_tuned_refocusing_phases,
    optimize_untuned_refocusing_phases,
    optimize_matched_refocusing_phases,
    optimize_ideal_v0crit_refocusing_phases,
    optimize_ideal_v0crit_excited_refocusing_phases,
    optimize_ideal_time_varying_refocusing_phases,
    run_tuned_refocusing_multistart,
    run_untuned_refocusing_multistart,
    run_matched_refocusing_multistart,
    run_ideal_v0crit_refocusing_multistart,
    run_ideal_v0crit_excited_refocusing_multistart,
    run_ideal_time_varying_refocusing_multistart,
    evaluate_tuned_excitation_pulse,
    evaluate_tuned_inverse_excitation_pulse,
    optimize_tuned_excitation_phases,
    optimize_tuned_inverse_excitation_phases,
    run_tuned_excitation_multistart,
    run_tuned_inverse_excitation_multistart,
    run_tuned_excitation_inverse_pipeline,
    multistart_to_matlab_results,
    save_multistart_results_npz,
    load_optimization_results,
    summarize_matlab_results,
    select_matlab_result_program,
    analyze_matlab_optimization_results,
    analyze_tuned_inverse_result_pair,
)

```

The optimization layer is useful for compact pulse-design studies and MATLAB-style result inspection. Broad historical `fmincon` parity and strong inverse-excitation cancellation parity remain validation targets.

10 Known Gaps

The Python port is intentionally conservative where MATLAB parity still matters. The largest remaining gaps are broader high-Q matched-probe validation, full historical MATLAB `.mat` structure parity, broad `fmincon` parity, paper-level scalar-coupling fixture comparisons, relaxation-aware coupled-spin pulse-sequence runners, and acceleration backends beyond the current NumPy implementation.

New work should add small reference fixtures first, then a NumPy implementation, then optional acceleration or plotting once numerical behavior is pinned down.